

System Calls - Worksheet

Part A: Code Analysis

Question 1: Analyze the following code and answer the questions:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child Process\n");
    } else {
        printf("Parent Process\n");
    }
    return 0;
}
```

- a) What system call is used in this program?
 - b) How many processes will be created when this program runs?
 - c) What will be the possible output of this program?
-

Question 2: File Operations Code

```
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("test.txt", O_RDONLY);
    char buffer[100];
    read(fd, buffer, 100);
    close(fd);
    return 0;
}
```

- a) List all the system calls used in this code.
- b) What does O_RDONLY flag indicate?
- c) What is the purpose of the file descriptor 'fd'?

Question 2: File Permissions

Explain the meaning of the following permission modes in the open() system call:

Flag	Meaning
O_RDONLY	
O_WRONLY	
O_RDWR	
O_CREAT	
O_APPEND	

Part B: Code Completion

Complete the following code snippets:

1. Write code to create a file named "output.txt" and write "Hello World" to it:

```
int fd = open(_____, _____);  
write(fd, _____, _____);  
close(fd);
```

2. Complete the fork() example to make the child process print its PID:

```
pid_t pid = fork();  
if (pid == 0) {  
    printf("Child PID: %d\n", _____);  
}
```

3. Fill in the system call to wait for a child process to terminate:

```
int status;  
_____(&status);
```

Part C: Error Handling

1. Why is error checking important when using system calls?

2. Modify the following code to include error checking:

```
int fd = open("file.txt", O_RDONLY);
read(fd, buffer, 100);
```

Part D: System Call Categories

Classify the following system calls into their appropriate categories:

Process Control	File Management	Information

System calls to classify: *fork()*, *open()*, *getpid()*, *read()*, *exec()*, *close()*, *wait()*, *getuid()*, *write()*, *exit()*

Part E: Process Management

Question 1: Fork and Exec

Explain the difference between *fork()* and *exec()* system calls. When would you use each?

Question 2: Process Hierarchy

Draw a process tree for the following code:

```
fork();
fork();
fork();
```

How many processes will be created in total?

Part F: Application Scenarios

For each scenario, identify which system call(s) would be most appropriate:

1. A shell program needs to execute a user command like "ls -l":
2. A program needs to know the current time and date:
3. A web server needs to handle multiple client connections simultaneously:
4. A program needs to allocate more memory during runtime:
5. A daemon process needs to run in the background:

Part G: Critical Thinking and Scenario based

1. Explain why system calls are more expensive (time-consuming) than regular function calls.
2. What security issues might arise if applications could directly access hardware without system calls?

Scenario : Web Server Request Handling

You are developing a simple web server that needs to handle multiple client connections simultaneously. When a client connects, the server should create a separate process to handle that client's request while the main server process continues to listen for new connections.

Questions:

- a) Which system call would you use to create a new process for each client connection? Explain how it works.
- b) How would the parent (main server) process know when a child process (handling a client) has finished?
- c) Write pseudocode or actual C code showing the basic structure of this server.
- d) What problem might occur if the parent process doesn't properly wait for terminated child processes?